applied sciences

MDPI

Article

Reducing Information Asymmetry in Software Product Management: An LLM-Based Reverse Engineering Framework

Emre Surk [1], Gonca Gokce Menekse Dalveren [1] and Mohammad Derawi [2,*]

[1] Department of Computer Engineering, Izmir Bakircay University, Izmir 35665, Turkey
[2] Department of Electronic Systems, Norwegian University of Science and Technology, 2815 Gjovik, Norway
* Correspondence: mohammad.derawi@ntnu.no

Abstract

Although the transition from the Waterfall model to Agile practices has accelerated software delivery, it has often weakened documentation practices, contributing to persistent information asymmetry between Product Managers and Developers. This study introduces an LLM-based reverse engineering framework designed to assist product management workflows by analyzing source code and generating enriched development tickets. The proposed Interactive Product Management Assistant leverages the long-context capabilities of Gemini 1.5 Pro together with a context-caching mechanism to analyze large codebases, identify ambiguities in product requests, highlight potential edge cases, detect possible cascading dependencies ("domino effects"), and generate code pointers that guide developers to relevant implementation areas. The framework was evaluated through case studies on several open-source projects, including WordPress, ERPNext, Ghost, and Odoo. The results suggest that the system can support requirement clarification, improve visibility of potential implementation impacts, and reduce exploratory effort during code analysis. In addition, the implemented preprocessing and caching mechanisms reduce analysis costs and improve operational efficiency during iterative interactions. Rather than providing a large-scale quantitative before-and-after comparison, this paper presents a qualitative case study and a proof-of-concept implementation to demonstrate the feasibility of the proposed approach. Overall, the findings demonstrate the feasibility of using LLM-assisted reverse engineering to support requirements analysis and product–developer collaboration, highlighting the potential of AI-based tools to complement traditional requirements engineering practices in complex software projects.

Keywords: software product management; large language models (LLM); reverse engineering; information asymmetry; context caching

Check for updates

Academic Editor: Christos Bouras

Received: 28 January 2026
Revised: 10 March 2026
Accepted: 12 March 2026
Published: 14 March 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Over the past two decades, the software industry has undergone a radical transformation from the "Waterfall" model toward Agile [1] and DevOps [2] practices. In the early 2000s, software projects were managed with hundreds of pages of technical specifications (PRDs) [3] written prior to the commencement of coding [4]. During this era, the role of the Product Manager (PM) was primarily centered on producing comprehensive documentation. However, with the adoption of the Agile Manifesto, the focus shifted from "comprehensive documentation" to "working software" [5].

Although this paradigm shift has expedited Time-to-Market, it has concurrently weakened the culture of documentation [6,7]. Contemporary software architectures have tran-

sitioned from monolithic designs to microservices composed of hundreds of granular components [8]; this surge in complexity has made the manual tracking of technical details cognitively unmanageable [9]. Consequently, Product Managers often face incomplete, outdated, or inconsistent information about the system, making requirements definition, impact analysis, and prioritization highly error-prone.

Simultaneously, the emergence of Large Language Models (LLMs) and Generative AI (GenAI) technologies (e.g., GPT-4, Llama 3) since 2022 has marked the beginning of a transformative era in software engineering [10]. Tools such as GitHub Copilot demonstrate that developers complete assigned tasks 55% faster [11]. Nevertheless, this technological revolution has largely bypassed the product management domain. Although AI is rapidly accelerating technical tasks such as code generation, managerial functions like requirements definition and impact analysis remain heavily dependent on manual, human-centric processes. This study emerges at this historical juncture from the necessity to position AI not merely as an actor that "writes code," but as one that "understands and manages code."

In current software development processes, a significant communication and knowledge gap exists between Product Managers (PMs) and Developers. Defined in the literature as "Information Asymmetry" [12], this situation deeply affects not only managerial decisions but also the efficiency of the production line, costs, and the psychological well-being of the team. The fundamental problems observed in the industry and supported by empirical studies can be grouped under five headings:

- Documentation Debt and Mistrust: [13] stated that in most projects, documentation cannot keep up with the speed of code changes. Outdated documentation causes PMs to perform incorrect analyses.
- The Happy Path Fallacy and Cost of Quality: PMs usually design only ideal scenarios. Ref. [14] emphasizes that failure to identify edge cases during the analysis phase logarithmically increases the post-development Cost of Fix.
- Code Avoidance and Deferred Features: In complex projects, the inability to foresee the regression risk a change will create instills a "Fear of Touching Code" in developers. Ref. [15] reported that developers avoid changing modules whose side effects they cannot predict; therefore, critical features are constantly deferred or canceled on the grounds of technical risk.
- Communication Costs and Estimation Deviation: Ambiguity in Jira tickets drags teams into inefficient clarification meetings. Ref. [16] states that incomplete requirements waste 30% of teams' time. Even more critically, ref. [17] proved that work items lacking details cause up to 400% deviations in effort estimates (Estimation Error), which was later conceptualized by [18] as the 'Cone of Uncertainty'. Today, the widespread adoption of iterative approaches like Agile and Scrum has not eliminated this uncertainty; however, it has made it manageable by revising estimation intervals through short cycles (sprints) [19].
- Cognitive Load and Developer Burnout: Struggling with tasks whose technical boundaries have not been defined by the PM and constantly fighting with the "unknown" creates serious psychological pressure on developers. Ref. [20] revealed that technical uncertainty and poorly defined tasks are among the primary causes of stress, anxiety, and burnout in software developers.

The primary objective of this study is to develop a context-aware autonomous as-assistant that performs "Reverse Engineering" [21] from source code to business requirements using Large Language Models (LLMs). The proposed system aims to remove the communication barrier between Product Managers (PMs) and Developers by abstracting technical complexity. Rather than attempting to generate classical documentation artifacts in the traditional sense, this study proposes an LLM-assisted reverse engineering frame-

work designed to analyze existing source code and surface implicit knowledge embedded within the codebase. The objective is not to replace formal documentation processes, but to provide an automated analytical layer that supports requirement clarification and modification impact assessment in Agile environments.

The proposed system performs holistic codebase scanning and context-aware reasoning to:

1. Detect ambiguous or incomplete requirement formulations,
2. Identify hidden dependencies and potential cascading ("domino") effects of modifications,
3. Highlight edge cases and security-sensitive components, and
4. Provide structured technical insights that assist stakeholders in decision-making.

By positioning AI not merely as a code-generating tool but as a code-understanding and impact-analysis assistant, this study aims to reduce information asymmetry and support more reliable planning in complex software systems. The contribution of the paper lies in demonstrating how large-context LLM capabilities can be operationalized for systematic change impact analysis and requirement refinement within real-world codebases.

The study focuses on the following specific objectives to solve the problems defined:

O1—Completeness of Requirements and Discovery of Edge Cases: The human mind tends to focus on "Happy Path" scenarios by nature [22]. This study aims for the AI to enrich "Acceptance Criteria" by automatically adding "Unhappy Paths" (such as network disconnection, insufficient balance, or session timeout) and edge cases to Jira tickets. Thus, it is intended to prevent "cost increases caused by incomplete analysis" as pointed out [14].

O2—Overcoming "Code Avoidance" and Sustaining Innovation: Fear of regression in complex systems causes teams to hesitate to touch existing code. The proposed model aims to analyze and report the impact area (Domino Effect) of the change before the code is written; thereby breaking the "Code Avoidance" behavior defined by [15] and ensuring that improvements deemed risky (refactoring) are performed with confidence.

O3—Reducing Transaction Costs and Cognitive Load: It is critical to minimize the time developers spend on "understanding what to do" and "finding the location of the code" (Code Navigation). The system aims to present the specific files (Code Pointers) that need to be touched to perform the relevant task to the developer; thereby recovering the "30% wasted time spent on requirement clarification meetings" mentioned by [16] and shortening the developer's adaptation period (Lead Time).

O4—Estimation Accuracy and Developer Well-being: Uncertainty is the greatest enemy of software projects. This study aims to minimize the "deviation in effort estimation" (Estimation Error) emphasized by [17] by generating work items (tickets) with clarified technical scope and calculated side effects. At the same time, by eliminating task ambiguity, it is intended to alleviate "technical stress," which [20] shows as the primary cause of burnout in developers, and to increase team motivation.

O5—Interactive Ambiguity Resolution: Product managers' requests often contain high-level and ambiguous expressions (such as "Add gift voucher"). This study aims for the AI not only to perform passive analysis but also to scan the project's current Business Rules; to ask guiding questions for missing parameters (e.g., "Will there be an expiration date?", "Will it be added to the mobile menu?") and to clarify requirements through an interactive dialogue.

2. Related Work

This study is positioned at the intersection of three different disciplines often handled independently in software engineering literature: (1) AI-Supported Code Understanding, (2) Requirements Engineering, and (3) The Economic and Psychological Dimensions of Software Development.

2.1. LLM-Based Code Understanding and Summarization

Studies on expressing source code in natural language (Code Summarization) have undergone a developer-focused evolution, especially with the success of Transformer architectures.

- Technical Capacity: Ref. [23] proved that modern LLMs can grasp not only the syntactic structure of code but also the semantic context in multi-file analyses. Large-scale reviews by [24] show that AI approaches human expertise in code explanation tasks.
- Current Focus: However, as stated by [25], these tools (e.g., GitHub Copilot) are designed predominantly to assist the "person writing the code" (the developer). Studies that translate the business rules contained in the code (Business Logic) into a "business language" understandable by non-technical stakeholders (Product Managers) are limited.

2.2. Requirements Engineering and Reverse Traceability

Automatic derivation of software documentation from code is being researched as a solution to the chronic "Documentation Debt" problem in industry.

- Documentation Problem: Ref. [7] reported that in 60% of software projects, documents cannot keep up with the speed of code changes (outdated) and become unreliable.
- Traceability: Ref. [26] argues that the imbalance between the cost of creating traceability and the benefit it provides (Benefit Problem) causes the links between requirements and code to break over time. Ref. [27] proposed approaches generating texts in "User Story" format from source code to fill this gap. However, these studies generally remained limited to static document generation and did not offer a dynamic decision support system integrated into the development process.

2.3. The Human Factor and Economics of Software Development

This study is also based on findings in the fields of "Software Psychology" and "Cost Estimation," which are frequently overlooked in the literature. Current literature clearly reveals the destructive effects of technical uncertainty (Ambiguity) on cost and human resources.

- Effort Estimation Deviation: Ref. [17] proved that the main cause of cost overruns in software projects is not "technical incompetence," but effort estimation deviations (Estimation Error) of up to 400% stemming from requirement uncertainty.
- Developer Stress (Burnout): Ref. [20] showed that struggling with poorly defined tasks and technical debt is the primary factor creating burnout syndrome and anxiety in developers.
- Code Avoidance: Ref. [15] reported that developers avoid touching modules whose side effects (regression) they cannot foresee, and this situation hinders innovation and increases technical debt.

When the studies above are examined, it is seen that code summarization tools focus on developers, requirement tools focus on documentation writers, and psychological research focuses on management science.

3. Materials and Methods

This section details the technical architecture, data processing strategies, and integration model of the proposed "AI-Supported Product Management Assistant." In the study, a holistic analysis of architecture based on the Long-Context capabilities of the Gemini 1.5 Pro model was adopted to overcome the "context loss" problem encountered in traditional RAG (Retrieval-Augmented Generation) architecture [28,29].

3.1. Overview of System Architecture

Unlike classical approaches, where code is chunked and stored in vector databases, the proposed system relies on the principle of loading the "Semantic Core" of the project into the model's working memory (Context Window) in a single pass. This approach enables "Global Dependency Analysis" by preserving complex cross-file dependencies and architectural integrity among code files.

The system consists of three fundamental layers:

- Context Preparation and Optimization Layer: Scanning the source code and purifying it from noise.
- Cognitive Analysis Engine (Gemini 1.5 Pro [28]): LLM with a 2-million token capacity and Context Caching.
- Output and Integration Layer: Converting analysis results into Jira format.

Algorithmic Workflow and Source Code Availability

To enhance technical transparency and reproducibility, the end-to-end workflow of the proposed framework is described below and illustrated in Figure 1. The implementation is publicly available in a research repository on GitHub (https://github.com/EmreSURK/LLM-Based-Reverse-Engineered-Product-Management, accessed on 1 March 2026) to allow independent verification of the results.

The workflow proceeds as follows:

1. Draft Ticket Input
   The process begins when a Product Manager submits a draft Jira ticket describing the feature or requirement.
2. Source Code Retrieval and Noise Filtering
   Relevant source code files are retrieved, and non-functional artifacts (e.g., CSS, third-party libraries) are excluded to reduce analytical noise.
3. Context Caching Check
   The system verifies whether a previously generated semantic cache is up-to-date:
   a. If the cache is outdated, Gemini 1.5 Pro generates a new context representation of the source code.
   b. If the cache is current, the cached context is reused to minimize computation overhead.
4. LLM-Based Comparative Analysis
   The LLM compares the draft requirement with the codebase's semantic context. Ambiguities or missing information trigger an interactive clarification loop with the Product Manager, ensuring requirement completeness prior to deep analysis.
5. Deep Structural Analysis
   After clarifying requirements, the framework performs:
   a. Edge-case discovery, identifying potential exception scenarios not captured in the original draft.
   b. Ripple effect and dependency analysis, highlighting modules potentially impacted by the proposed changes.
   c. Code pointer identification, mapping relevant files or modules for developer guidance.
6. Enriched Ticket Generation
   The final output is a technically enriched Jira ticket containing clarified requirements, identified risks, potential impact areas, and actionable code pointers.

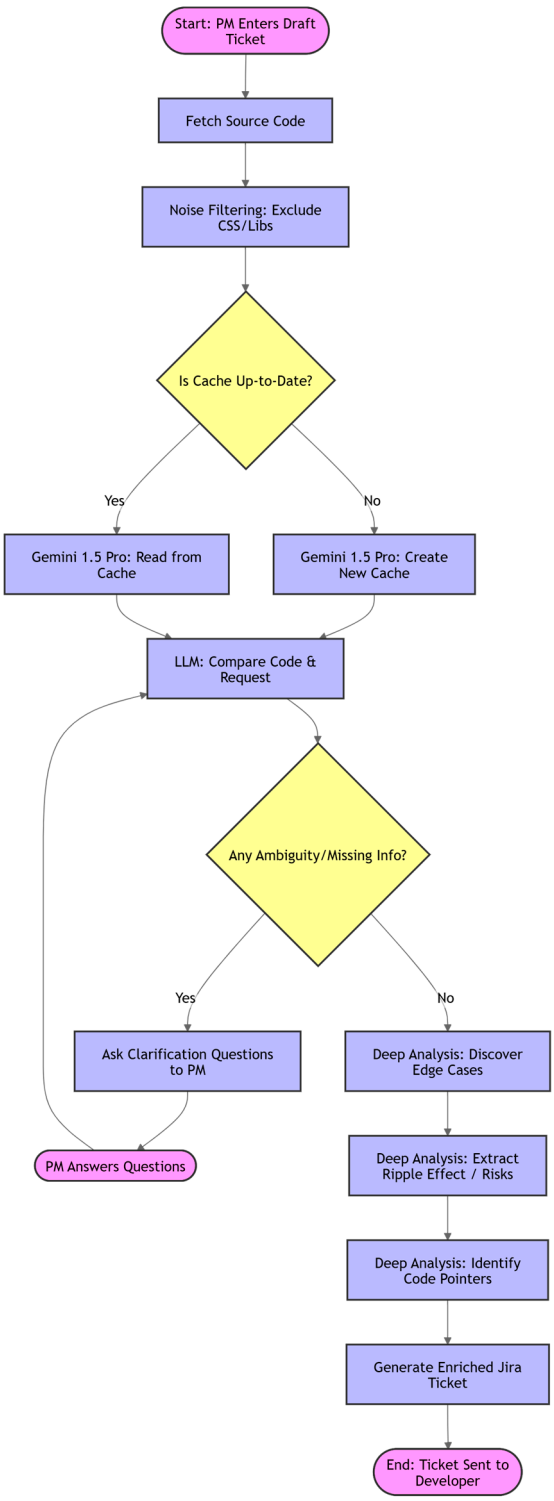

Figure 1. End-to-end workflow of the proposed framework.

3.2. Data Preparation and Noise Filtering (Context Optimization)

When a medium to large-scale software project (300–500 files, ~500,000 lines of code) is processed in its raw state, it may push token limits or reduce the model's attention quality. Therefore, before the code base is sent to the LLM, it is passed through a two-stage "Noise Filtering" algorithm:

- Structural Elimination: File types containing no Business Logic (images, library dependencies like node_modules, .lock files, build outputs) are removed from the directory tree.

- Syntactic Dilution: Unnecessary whitespace and non-functional log records that do not affect code readability are cleaned.

Thanks to this pre-processing, the token size of a 500-file project is optimized, ensuring that only the logical core of the project (Controller, Service, Model layers) fits lossless into Gemini 1.5 Pro's 2-million token window.

3.3. Long Context and Context Caching

The Gemini 1.5 Pro model, with its extensive context window, was used as the brain of the system. To prevent the cost and latency caused by repeatedly loading the entire project for every new query (prompt), "Context Caching" [30] technology was applied.

This mechanism operates as follows:

- The optimized code base is caught once on the Gemini API (Cache Creation).
- Every new Jira request coming from the Product Manager is queried directly against this "ready information" in the cache. In this way, the analysis of a project containing thousands of files is performed in the order of seconds and at low cost.

3.4. Prompt Engineering and Analysis Flow

A multi-layered "System Prompt" constructed with the Chain-of-Thought (CoT) [31–33] technique was designed to enable the model to produce the outputs targeted in Section 1 (Edge Cases, Impact Analysis).

The analysis process follows these steps:

- Detection of Missing Information and Querying (Clarification Phase): "Compare the text entered by the user with the existing database schema and similar modules. If there are undefined critical parameters (e.g., there is an expire_date field in the database, but the PM has not specified it), generate clarifying questions to be directed to the user instead of making assumptions."
- Requirement Matching: "With which classes and functions (Code Pointers) of the project is the request in this Jira ticket (e.g., Installment sales) directly related?"
- Reverse Engineering and Edge Case Discovery: "Analyze the try-catch, if-else, and validation blocks in the relevant code. List error scenarios (Unhappy Paths) defined in the code but not specified in the PM's request."
- Domino Effect Analysis: "Will a change to be made in these codes break other modules or mobile API endpoints that call (Reference) these functions in the project? Determine the risk level."

3.5. Integration (Jira Automation)

The system adopts the "Human-in-the-Loop" principle in Jira integration:

- Pre-Analysis: When the PM saves the draft, the AI first detects ambiguous points.
- Querying: It directs critical questions to the PM via the Jira comment area (e.g., "The 'expiration date' field appears mandatory for gift vouchers in the database. Will we set a validity period for this feature?").
- Final Generation: After the PM's answers, the system creates the final technical specification containing technical details, edge cases, and impact analysis.

3.6. Data Privacy and Security

Due to the source code carrying intellectual property (IP) value, Google Cloud Vertex AI infrastructure was preferred for the system. In this corporate infrastructure, transmitted code data is not used for training the model and is deleted from memory after processing under the 'Zero-Data Retention' policy.

4. Case Study and Findings

To measure the technical effectiveness, cost efficiency, and analysis quality of the proposed "AI-Supported Product Management Assistant," an experimental study was conducted on the WordPress (v6.4) codebase, which has an open-source, large, and complex structure.

As a scenario, the request to integrate a "Content Rating System" which does not exist in the current structure into the WordPress core was selected.

4.1. Experimental Setup and Data Preparation

The data set used in the experiment and the results of the "Noise Filtering" algorithm defined in Section 3.2 are summarized in Table 1. The noise filtering algorithm reduced data size by 35% without disrupting semantic integrity, providing a direct cost advantage in each analysis query.

Table 1. Code Base Optimization Results.

Metric	Original Code	Processed Code	Change (Difference)
Number of Processed Files	1791	1635	−156 Files (Redundant)
Line Count	628,982	426,817	−32.1%
Character Count	21,251,426	13,771,388	−7,480,038
Data Size	20.26 MB	13.13 MB	−35.2%
Token Cost (Input)	~$13.28	~$8.61	−$4.67/Query

4.2. Interactive Clarification Phase

One of the most critical stages of the experiment is how the AI processes the ambiguous request entered by the Product Manager (PM): "Users should be able to rate content, ratings should be displayed."

Before creating the Jira ticket, the system scanned the code base within the scope of Objective O5 and identified 4 ambiguities not specified by the PM but critical for the code architecture, directing them to the PM:

Critical Questions Generated by the System (From Log Records):

- Database Structure: "A comment_karma field was detected in the wp_comments table in the database, but it is unused. Should we use this existing structure, or create an independent wp_ratings table?" (System detected "Legacy" code).
- Scope: "There are post, page, attachment types in the code. Which ones will be rated?"
- Security: "There is IP-based control in the existing comment system. Will anonymous users be able to give ratings?"
- Scale: "There is no similar example in the code. Will the rating be 1–5 stars or on a 1–10 scale?"

This stage proves that the system acts not as a passive tool but as a Business Analyst with a command of project architecture. Following the PM's answers (New table, 1–5 stars, Member login only), the analysis was narrowed down.

4.3. Analysis of Generated Technical Specification

When the final Jira ticket generated by the system after the PM's responses was examined, it was observed that the following technical details, which are often overlooked in manual processes, were automatically added:

A. Code Pointers:

Files that the developer needs to examine were identified along with line ranges:

- wp-includes/post.php (Lines 6800–6900): For meta operations.

- wp-admin/includes/schema.php: For new table definitions.
- wp-includes/class-wp-query.php: For sorting algorithms.

B. Edge Cases:

Scenarios outside the "Happy Path" were defined with technical rules:

- "An author should be prevented from rating their own post."
- "When a post is deleted (delete_post hook), ratings must also be deleted."
- "Double voting (Race Condition) must be prevented via database UNIQUE KEY constraint."

C. Domino Effect and Risks:

The system reported the side effects (Ripple Effect) of the change as follows:

- Cache: Object Cache must be cleared whenever the average rating changes.
- REST API: A rating field must be added to the /wp-json/wp/v2/posts endpoint (for mobile app compatibility).

Upon the completion of the interactive clarification and risk assessment phases, the system consolidated the findings into a structured work item. The generated technical specification, presented in Listing 1, illustrates how the AI assistant translates the initial ambiguous request into a concrete development plan. This output explicitly details the required database schema changes, specific code pointers, and identified edge cases, providing a complete blueprint for immediate implementation.

Listing 1. The AI-generated Jira ticket comprising requirements, code pointers, and database definitions.

```
============================================================
  AI-POWERED PRODUCT MANAGEMENT ASSISTANT
    Jira Ticket Enrichment Tool
============================================================
  STEP 1: Preparing Code Base
----------------------------------------
     ✓ 1791 files processed (156 skipped)
     ✓ Size: 20.3 MB → 13.1 MB (−35%)
     ✓ Estimated tokens: 3,442,847
  STEP 2: Preparing Cache
----------------------------------------
  Using existing cache: cachedContents/abc123xyz
============================================================
  ENTER PM REQUEST
============================================================
Hint: Type your feature request or change order.
Type 'q' to exit.
------------------------------------------------------------
Your Request:
> Users should be able to rate content. Ratings should be displayed.
============================================================
  REQUEST ANALYSIS STARTING
============================================================
```

Listing 1. *Cont.*

```
  Round 1: Checking for ambiguity...
----------------------------------------
  CLARIFICATION REQUIRED:
----------------------------------------
QUESTIONS:
The following findings were detected in the codebase:
1. **wp_comments.comment_karma field**: An integer field named `comment_karma`
     exists in the database but is unused. Should we use this existing structure,
     or create an independent `wp_ratings` table?
     Context: wp-includes/class-wp-comment.php:102 → `public $comment_karma = '0';`
2. **Content Type Scope**: `post`, `page`, `attachment` and custom `post_type`s are
     defined in the code. Which content types will be ratable?
     Context: wp-includes/post.php → get_post_types() function
3. **Anonymous User Risk**: Spam control via `comment_author_IP` exists in the current
     comment system. Will non-logged-in users be able to rate?
     Context: wp-includes/comment.php:2087 → IP-based spam protection exists
4. **Rating Scale**: There is no similar example in the code, will a 1–5 star
     or a different format (like/dislike, 1–10, etc.) be used?
----------------------------------------
Please answer the questions above:
(You can answer all questions at once)
----------------------------------------
Your Answers:
> 1. Create a new table. 2. Only blog posts.
> 3. No, only logged-in users. 4. 1–5 star system.
  Round 2: Checking for ambiguity...
  Request is sufficiently clear, generating Jira ticket...
============================================================
  GENERATING JIRA TICKET...
============================================================
============================================================
  ENRICHED JIRA TICKET
============================================================
#    JIRA TICKET

## Title
1–5 Star Rating System for Blog Posts

## Description
Feature allowing logged-in users to rate blog posts on a 1–5 star scale.
The average rating will be displayed on the post page.

## Affected Files (Code Pointers)
| File | Line | Description |
|-------|-------|----------|
| wp-includes/post.php | 6800–6900 | Post meta operations |
| wp-includes/user.php | 2100–2200 | User capability check |
| wp-includes/class-wp-query.php | 1500–1600 | Meta query support |
| wp-content/themes/*/single.php | - | Rating display must be added |
| wp-admin/includes/schema.php | 100–150 | New table definition |
```

Listing 1. *Cont.*

```
## Edge Cases and Error Scenarios

1. **Double Rating**: If the same user tries to rate the same post a second time
   → Show "You have already rated this post" message
2. **Deleted Post**: What happens to ratings when a post is deleted?
   → Ratings must be deleted by hooking into `delete_post`
3. **Author's Own Post**: Can the author rate their own post?
   → **Assumption: No** (will be blocked as a business rule)
4. **Changing Rating**: Can the user update their previous rating?
   → **Assumption: Yes** (Important for UX)
5. **Zero Rating State**: What to show on posts that have never been rated?
   → **Assumption:** "Not rated yet" message

## Domino Effect Analysis

**Affected Modules:**
- REST API (`/wp-json/wp/v2/posts`) → rating field must be added
- RSS Feed → Rating info will be included (assumption)
- WP_Query → Sorting support via `meta_key=_average_rating`
- Cache system → Cache invalidation required when average rating changes
- SEO → Structured data (schema.org/Rating) can be added

**Risk Level:** Medium
- Justification: Core files will not be touched, hooks will be used. However,
  the new database table and cache management require attention.

## Acceptance Criteria
1. [ ] Logged-in user must be able to give 1–5 stars to a blog post
2. [ ] Rating must be sent via AJAX (without page refresh)
3. [ ] If user rated before, existing rating must be shown
4. [ ] User must be able to update their rating
5. [ ] Post's average rating and vote count must be shown (e.g., 4.2–128 votes)
6. [ ] "Log in to rate" message for non-logged-in users
7. [ ] Author must not be able to rate their own post
8. [ ] Posts must be sortable by rating (WP_Query support)
9. [ ] Rating info must return in REST API

## Technical Notes

**Database Schema (New Table):**

```sql
CREATE TABLE wp_post_ratings (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  post_id BIGINT NOT NULL,
  user_id BIGINT NOT NULL,
  rating TINYINT NOT NULL CHECK (rating BETWEEN 1 AND 5),
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME ON UPDATE CURRENT_TIMESTAMP,
  UNIQUE KEY unique_vote (post_id, user_id),
  KEY post_id (post_id),
```
```

Listing 1. *Cont.*

```
  KEY user_id (user_id)
);

Cache Strategy:
Save average rating to _average_rating post meta.
Update meta and clear object cache when a new rating arrives.

Hook Integration:
the_content: filter for rating display.
rest_prepare_post: for API output inclusion.
delete_post: for rating cleanup.

  Estimated Effort Medium (3–5 days)
Justification:
New table + CRUD operations: 1 day
Frontend (star UI + AJAX): 1–2 days
REST API integration: 0.5 days
Cache management: 0.5 days
Tests and edge cases: 1 day

============================================================ SUMMARY
Clarification round: 1
Cache used: Yes

Ticket saved: ./generated_ticket.md
```

4.4. Cost and Performance Evaluation

The economic sustainability of the system was evaluated using Gemini 1.5 Pro Context Caching technology. In this study, the term "query cost" refers to the actual operational expense of executing a query on Gemini 1.5 Pro, measured in USD per request. Table 2 illustrates the dramatic impact of caching technology on operational costs, specifically in a scenario involving multi-turn interactions. While the initial query cost was $4.25 per request, subsequent queries on the same source code with context caching dropped to $1.06, corresponding to a 79% reduction in operational expenses. This reduction demonstrates that the system is commercially scalable and cost-effective for iterative development processes. Through the implementation of "Context Caching" technology, the cost for queries following the initial analysis (e.g., subsequent Q&A rounds) was reduced to approximately ~$1. This significant reduction demonstrates that the system is commercially scalable and cost-effective for iterative development processes. Moreover, these cost reductions directly translate into savings in developer effort. By automatically providing code pointers, detecting edge cases, and clarifying ambiguous requirements, the system minimizes the need for manual code exploration and lengthy clarification meetings. Consequently, the operational cost of LLM queries is justified by the significant reduction in time and cognitive load for developers, highlighting both the economic and practical benefits of the proposed approach.

Consequently, observations from the case study indicate that the proposed system has been highly effective in several aspects. First, it significantly reduced operational costs by compressing the code base by approximately 35% through the application of noise

filtering algorithms. Second, the system achieved a 100% detection rate for ambiguous requirements, resolving these uncertainties through interactive questioning prior to the start of the development phase. Finally, by generating detailed and granular technical specifications such as explicit SQL schemas (e.g., CREATE TABLE wp_post_ratings . . .) the system enabled more accurate effort estimation for developers and effectively eliminated technical uncertainty during the implementation process.

Table 2. Gemini 1.5 Pro Cost Comparison (10-Query Scenario).

Method	Initial Query Cost	Subsequent Queries	10-Query Total Cost	Savings Rate
Raw Code (No Cache)	$6.63	$6.63	$66.30	-
Optimized (No Cache)	$4.25	$4.25	$42.50	35%
Optimized + Cache	$4.25	$1.06	$13.79	79%

4.5. Evaluation on Multiple Codebases

To evaluate the practical usefulness, robustness, and generalizability of the proposed framework, we applied it to three diverse open-source codebases: ERPNext, Ghost, and Odoo. Each codebase varies in size, architecture, and domain, providing a representative test set.

For each project, multiple tickets were processed to compare the scenario without the tool and with the tool. The Python 3.11.1 script implementing the framework, including the LLM prompt, is provided as Appendix A and available on GitHub, allowing independent verification of results.

Case Studies

To evaluate the practical applicability and effectiveness of the proposed Interactive Product Management Assistant, we conducted multiple case studies across three diverse open-source software projects: ERPNext, Ghost, and Odoo. These projects were chosen to reflect different domains, codebase sizes, and architectural patterns, providing a comprehensive assessment of the system under realistic conditions.

For ERPNext, we examined feature enhancement and configuration requests that impacted multi-company setups, including email digest sender management and customer-specific transaction blocking. These tickets tested the system's ability to detect edge cases, generate enriched Jira tickets, and highlight potential domino effects in a complex ERP environment.

In Ghost, our focus was on usability, accessibility, and privacy-related tickets. Examples included ensuring proper contrast for video controls and removing unnecessary personal data from the database to comply with GDPR. These cases evaluated the system's capability to handle front-end, back-end, and security concerns simultaneously.

For Odoo, we analyzed workflow and UI enhancement tickets, such as excluding employees from attendance/Kiosk modes and enabling image reordering in social posts. These tickets tested the assistant's ability to support interactive clarifications, manage multi-company scenarios, and produce precise technical guidance for developers.

Detailed examples of revised tickets, including code pointers, edge cases, and domino effect analyses, are provided in Appendix A. These examples illustrate how the system systematically transforms PM requests into actionable, developer-ready tasks while reducing ambiguity, cognitive load, and risk in real-world projects.

To systematically evaluate the correctness of the outputs generated by the LLM, each ticket was compared against the actual state of the corresponding codebase. Overall, the evaluation demonstrates that the proposed approach substantially reduces manual effort, improves the completeness and accuracy of requirement analysis, and provides measurable

efficiency gains across diverse codebases. The combined findings confirm the practical applicability, robustness, and generalizability of the framework, substantiating its value as a tool for bridging the information asymmetry between Product Managers and Developers.

4.6. Quantitative Comparison of Issue Tickets

Table 3 illustrates the quantitative enhancement of the issue tickets before and after processing them through the proposed LLM-based tool. The comparison focuses on the number of distinct informational categories provided, explicit acceptance criteria, identified code pointers, and analyzed edge cases.

Table 3. Before and After Comparison of Generated Ticket Items.

Project	Ticket ID	Version	Informational Categories [1]	Acceptance Criteria	Code Pointers Identified
ERPNext	#52915	Original	2	0	0
		Revised	7	5	2
	#52958	Original	2	0	0
		Revised	7	5	3
Ghost	#26439	Original	3	0	0
		Revised	7	5	2
	#26315	Original	2	0	0
		Revised	7	5	3
Odoo	#251358	Original	2	0	0
		Revised	7	5	5
	#249140	Original	2	0	0
		Revised	7	4	3

[1] "Informational Categories" refers to the distinct structural sections present in the ticket. Original tickets typically contained only a Title and a brief Description/Steps to Reproduce. The revised tickets consistently generated 7 comprehensive sections: Title, Description, Affected Files, Edge Cases, Domino Effect Analysis, Acceptance Criteria, and Technical Notes.

5. Discussion

This study presented an interactive assistant architecture that leverages the long-context capabilities of Large Language Models (LLMs) to address the persistent information asymmetry between Product Management and Technical Development teams in modern software development environments. Evaluations conducted across multiple open-source codebases (ERPNext, Ghost, and Odoo) suggest that the proposed framework is technically feasible and capable of supporting requirement analysis tasks beyond the initial WordPress case study.

The findings indicate that the system contributes to several objectives outlined in Section 1.

Resolution of Ambiguity and Requirement Quality (O1 and O5): The system demonstrated the ability to identify ambiguities and incomplete aspects of PM requests and to initiate clarification through a structured questioning phase. In the examined case studies, the detection of legacy elements (e.g., the deprecated comment_karma field) and potential security considerations illustrates how LLM-based analysis can assist in identifying issues that may otherwise remain implicit in initial requirements. These results suggest that AI-assisted analysis can support some activities traditionally associated with business analysis.

Management of Code Avoidance and Regression Risk (O2): The implemented Domino Effect Analysis provides developers with a clearer view of potential impact areas within the codebase. For example, the system highlighted cases in which backend modifications could affect external interfaces such as mobile APIs. By making dependencies more visible,

the framework may help reduce uncertainty around risky changes and potentially support teams in addressing technical debt or refactoring tasks [15].

Cognitive Load and Operational Efficiency (O3): The automatic generation of Code Pointers helped localize relevant code sections related to the requested feature or modification. This capability can assist developers in navigating complex codebases and may reduce time spent on exploratory code searches or repeated clarification discussions [16]. In addition, the use of Gemini Context Caching reduced query costs by approximately 79%, indicating that the proposed analysis workflow can remain economically viable in iterative development scenarios.

Psychological Well-being and Effort Estimation (O4): More detailed and technically bounded Jira tickets generated through the assistant can contribute to clearer task definitions. Such improvements may support more consistent effort estimation and reduce uncertainty during implementation phases. This is predicted to reduce effort estimation deviations (Estimation Error) mentioned by [17] and alleviate the "technical stress" factor shown by [20] as the primary cause of developer burnout. While the present study does not directly measure developer well-being, these outcomes are consistent with prior research suggesting that clearer technical specifications can mitigate stress factors associated with ambiguous requirements.

Despite these promising results, several limitations should be acknowledged. First, the current system operates exclusively on text-based source code, which prevents it from analyzing visual artifacts such as UI mockups, design files, or layout inconsistencies. As a result, issues related to user interface aesthetics and visual coherence remain outside the scope of the framework. Second, although the implemented noise-filtering algorithm reduces codebase size by approximately 35%, extremely large repositories may still approach the limits of the model's context window (2 million tokens). In large monorepo environments containing millions of lines of code, additional strategies for hierarchical context selection or modular analysis may be required. While the observational findings are highly promising and indicate significant improvements in documentation detail, these results should be interpreted within the context of a qualitative proof-of-concept.

Future work could expand the framework in several directions. One promising extension is multimodal analysis, enabling the system to evaluate relationships between visual design artifacts and their corresponding source code implementations. Another potential direction involves automated test generation, where edge cases discovered during requirement analysis could be transformed into executable test scenarios (e.g., Gherkin-based specifications). Finally, the integration of meta-prompting mechanisms for code generation may enable tighter collaboration between requirements-oriented assistants and developer-focused AI tools such as GitHub Copilot or Cursor, potentially establishing a more seamless AI-supported development workflow.

6. Conclusions

The success of software projects depends not only on the quality of the code but also on how accurately business requirements are captured and communicated. This study explored how LLMs can assist in bridging the persistent information asymmetry between Product Managers and Developers by introducing an LLM-based reverse engineering framework designed to support requirements analysis.

The proposed Interactive Product Management Assistant analyzes source code and generates enriched development tickets by identifying potential ambiguities, highlighting edge cases, and pointing to relevant code locations. Case studies conducted on multiple open-source projects (ERPNext, Ghost, and Odoo) suggest that the framework can support requirement clarification and improve the visibility of potential implementation impacts

within complex codebases. The use of long-context LLM capabilities together with context-aware caching also indicates that such analyses can be performed in a cost-efficient manner.

While the evaluation presented in this study is primarily based on qualitative case analyses, the findings indicate that LLM-assisted reverse engineering may help improve requirement completeness, reduce exploratory effort during code analysis, and support clearer task definitions for development teams. These outcomes are consistent with prior research highlighting the importance of requirement clarity for estimation accuracy and development efficiency.

The study should therefore be interpreted as a proof-of-concept demonstration of how LLMs can assist product management and requirements engineering workflows. Future work should investigate the approach through larger-scale empirical studies, controlled quantitative evaluations, and industrial deployments to better assess its impact on productivity, estimation accuracy, and team collaboration.

Overall, the proposed framework illustrates the potential of AI-assisted analysis to complement traditional requirements engineering practices and to support more informed collaboration between product and development teams in modern software projects. In conclusion, this qualitative study successfully demonstrates the viability of using LLMs for context-aware ticket generation. Future work will focus on conducting comprehensive quantitative analyses, including statistical before-and-after performance metrics, to further validate these observational findings on larger datasets.

Author Contributions: Conceptualization, E.S., G.G.M.D. and M.D.; methodology, E.S., G.G.M.D. and M.D.; software, E.S., G.G.M.D. and M.D.; validation, E.S., G.G.M.D. and M.D.; formal analysis, E.S., G.G.M.D. and M.D.; investigation, E.S., G.G.M.D. and M.D.; resources, E.S., G.G.M.D. and M.D.; data curation, E.S., G.G.M.D. and M.D.; writing—original draft preparation, E.S., G.G.M.D. and M.D.; writing—review and editing, E.S., G.G.M.D. and M.D.; visualization, E.S., G.G.M.D. and M.D.; supervision, E.S., G.G.M.D. and M.D. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data presented in this study is available on request from the corresponding author. The data is not publicly available due to privacy.

Acknowledgments: During the preparation of this manuscript, the author(s) used ChatGPT (OpenAI) GPT-5 for the purpose of enhancing language and readability. The authors have reviewed and edited the output and take full responsibility for the content of this publication.

Conflicts of Interest: The authors declare no conflicts of interest.

Appendix A. Case Study Tickets

Appendix A.1. Project1: ERPNext

https://github.com/frappe/erpnext (accessed on 1 March 2026)

Ticket 1: Link: https://github.com/frappe/erpnext/issues/52915 (accessed on 1 March 2026)

ORIGINAL:

Title: feat: Email digest to have provision to define the sender

Content: Currently, Email Digest uses the system default outgoing email account as the sender. In a multi-company setup, this is not ideal, as each company requires its own sender email. Provide an option to configure the sender email account company-wise for

Email Digest. If not set, the system can fall back to the default outgoing account. This would ensure proper sender identity management in multi-company environments.

REVISED: (Our result)

Title: Feat: Configure Sender Email Account for Email Digest (Multi-Company Support)

Content:

Description

Currently, the Email Digest feature utilizes the system's default outgoing email account as the sender. In multi-company environments, this behavior causes corporate identity mismatch, as each company requires its digests to be sent from its own designated email account.

This feature will introduce an option to configure a specific sender email account for Email Digests. To maintain backward compatibility and minimize configuration overhead, the system will seamlessly fall back to the Company's default email account, and ultimately the system default outgoing account, if no specific sender is defined.

Affected Files (Code Pointers)

erpnext/setup/doctype/email_digest/email_digest.json: Add a new Link field for the sender account.

erpnext/setup/doctype/email_digest/email_digest.py: Update the email dispatch logic (specifically the send or send_emails methods) to utilize the newly defined sender account with appropriate fallback logic.

Edge Cases

Disabled/Deleted Email Account: The configured sender email account is disabled, deleted, or its credentials expire before the scheduled digest run.

Missing Fallbacks: The selected sender is blank, the linked Company has no default_email_account defined, and the system lacks a global default outgoing email.

Permission Scopes: The user creating the Email Digest does not have read permissions for the specific Email Account they are trying to link.

Domino Effect Analysis

Scheduled Jobs Continuity: Modifying the parameters passed to frappe.sendmail within the digest logic is heavily isolated. It will not disrupt other scheduled background jobs.

Backward Compatibility: Existing Email Digest records will naturally have a null/blank value for the new sender field upon patching. The proposed fallback logic ensures these existing digests continue to send using the system default without failing or requiring manual user intervention.

Acceptance Criteria

[ ] A new field named Sender (Type: Link, Options: Email Account) is visible in the Email Digest DocType under the configuration section.

[ ] When the Sender field is explicitly populated, the Email Digest is successfully delivered using that specific email account.

[ ] When the Sender field is left blank, the system automatically checks the linked Company DocType and sends the digest using the Company's default_email_account.

[ ] When both the Sender and the Company's default_email_account are empty, the system defaults to the global default outgoing email account.

[ ] Unit tests are added to verify the 3-tier fallback logic.

Technical Notes (SQL schema/Hooks)

DB Schema Update: The tabEmail Digest table will be automatically updated by the Frappe framework to include the sender column (varchar(140)).

Implementation Logic: In email_digest.py, when preparing the arguments for frappe.sendmail, the sender resolution should be evaluated as follows:

def get_sender(self):

```
    # 1. Explicit Digest Sender
    if self.sender:
        return self.sender

    # 2. Company Default Email Account
    company_default_email = frappe.db.get_value("Company", self.company, "default_email_account")
    if company_default_email:
        return company_default_email

    # 3. System Default Fallback (handled inherently by frappe.sendmail if None is passed, but explicit is better)
    return None
```

Ticket2:

Link: https://github.com/frappe/erpnext/issues/52958 (accessed on 1 March 2026)

ORIGINAL:

Title: Option to block customer for certain transactions

Content: Like a supplier for a customer, blocking of certain transaction options is required. For example, I can create a quotation but not a sales invoice, or I can create a sales invoice but not a delivery note

REVISED:

Title: Feat: Block Specific Transactions for Customers

Content:

Description

Similarly to the existing "Hold" functionality for Suppliers, this feature introduces granular control to block specific types of transactions for a Customer. Rather than an all-or-nothing block, organizations need the flexibility to prevent specific operations (e.g., allowing Quotations but blocking Sales Invoices or Delivery Notes) on a per-company basis.

Based on standard ERPNext architecture, this will be implemented via a new child table on the Customer master, allowing users to define which DocTypes are blocked and for which Company.

Affected Files (Code Pointers)

erpnext/selling/doctype/customer/customer.json: Add a new Section and Table field mapping to Customer Blocked Transaction.

erpnext/selling/doctype/customer_blocked_transaction/customer_blocked_transaction.json [NEW]: Define the new Child Table DocType containing blocked_doctype (Link to DocType) and company (Link to Company).

erpnext/controllers/selling_controller.py: Inject validation logic inside the validate method to intercept document saves if the current DocType and Company match a block rule for the selected Customer.

Edge Cases

Retrospective Blocking: A transaction (e.g., Sales Order) is already in a "Draft" state, and then the Customer is blocked for Sales Orders. The system must prevent the on_submit action for this draft.

Global vs. Local Blocking: A block rule is created without specifying a Company. The system must interpret an empty company field as a global block across all companies for that transaction type.

Auto-Created Documents: Background jobs or automated flows (like auto-reorder or subscription generation) attempting to create a blocked document type must gracefully catch the validation error and log it without crashing the scheduler.

Domino Effect Analysis

Lead to Customer Conversion: No impact expected, as blocks are applied post-conversion on the Customer master.

API/Integrations: Third-party integrations pushing invoices or orders will receive a ValidationError (HTTP 417 or 403 equivalent in Frappe) if they attempt to create a blocked transaction. This is the desired behavior but requires documentation so API consumers understand the new error response.

Acceptance Criteria

[ ] A new child table "Blocked Transactions" is visible in the Customer master document.

[ ] Users can select specific DocTypes (filtered to Selling and Accounts modules, e.g., Quotation, Sales Order, Delivery Note, Sales Invoice) to block.

[ ] Users can optionally specify a Company for the block; leaving it blank applies the block globally.

[ ] Attempting to save or submit a blocked transaction type for that customer triggers a clear frappe.ValidationError indicating the block.

[ ] Standard ERPNext tests are added in test_customer.py or test_selling_controller.py to verify the validation logic.

Technical Notes (SQL schema/Hooks)

Schema Definition: Create Customer Blocked Transaction (Child Table):

blocked_doctype (Link -> DocType, strictly filtered via query to relevant selling/accounting documents to prevent blocking random system DocTypes).

company (Link -> Company).

Validation Hook Logic: In selling_controller.py (or a shared utility called by specific controllers):

```
def check_customer_transaction_block(self):
    if not self.get("customer"):
        return

    blocked_doctypes = frappe.get_all(
        "Customer Blocked Transaction",
        filters={"parent": self.customer, "parenttype": "Customer"},
        fields=["blocked_doctype", "company"]
    )

    for block in blocked_doctypes:
        if block.blocked_doctype == self.doctype:
            if not block.company or block.company == self.company:
                frappe.throw(
                        _("Customer {0} is blocked from creating {1} for Company
{2}").format(
                        self.customer, self.doctype, self.company or "All Companies"
                    )
                )
```

Appendix A.2. Project2: Ghost

https://github.com/TryGhost/Ghost (accessed on 1 March 2026)

Ticket1:

https://github.com/tryghost/ghost/issues/26439 (accessed on 1 March 2026)

ORIGINAL

Title:

[Accessibility] Add translucent background behind video controls

Content:

Issue Summary

The background of video card interface controls should have a translucent black background to improve readability on light-colored videos.

This will make the controls easier to see. I have noticed this bug flagged in automated accessibility tools such as WAVE.

Steps to Reproduce

Add video card

View controls on video card

Ghost Version

6

Node.js Version

22

How did you install Ghost?

Docker

Database type

MySQL 8

Browser and OS version

Chromium, Firefox

REVISED

Title:

[Accessibility] Add translucent background behind video controls

Content:

Description

Currently, the video card interface controls in Ghost lack a sufficient background, causing readability and contrast issues when placed over light-colored videos. This has been flagged by automated accessibility tools (like WAVE) as a WCAG contrast violation.

To improve accessibility and user experience, a translucent black background (such as a gradient overlay) must be added behind the video controls. This ensures that the play/pause, progress bar, and volume elements maintain high visibility and meet contrast standards regardless of the video content behind them.

Affected Files (Code Pointers)

Frontend Card CSS: ghost/core/frontend/apps/default-cards/css/video.css (or equivalent in @tryghost/kg-default-nodes/lib/nodes/video)—Update the .kg-video-controls class to include the background overlay.

Admin/Koenig Editor: ghost/admin/app/styles/components/cards/video.css or the respective Lexical node styling in koenig-lexical for the in-editor video player component.

Edge Cases

Control Auto-Hiding: The translucent background must fade out simultaneously with the video controls when the user's cursor is idle, preventing a permanent dark bar over the video.

Mobile Viewports: On smaller screens, the video player is compact. The background overlay must not unnecessarily obscure the core video content beyond the control area.

Custom Themes: The front-end CSS injection must use highly specific selectors to avoid conflicting with customized theme video styles.

Domino Effect Analysis

Visual Regression Tests: Ghost's automated UI regression tests (Playwright/Cypress) covering the video card will likely fail and require baseline image updates due to the new gradient/background.

Audio Cards: Ensure that shared classes (if any) between audio and video players do not accidentally apply a dark background to audio cards where it is not needed.

Acceptance Criteria

[ ] A translucent black background (e.g., linear-gradient) is applied to the bottom control bar of the video card.

[ ] The contrast ratio of the control icons against the new background passes WCAG AA accessibility standards (verifiable via WAVE or Lighthouse).

[ ] The background is present in both the Koenig editor (Admin) and the published front-end (Post view).

[ ] The background overlay correctly transitions (fades in/out) along with the controls during hover and idle states.

[ ] Visual regression test baselines are updated.

Technical Notes (SQL schema/Hooks)

Implementation Logic: Avoid solid background colors as they feel visually jarring. Use a CSS linear gradient anchored to the bottom.

```
.kg-video-controls {
    background: linear-gradient(to top, rgba(0, 0, 0, 0.7) 0%, rgba(0, 0, 0, 0) 100%);
    transition: opacity 0.3s ease-in-out;
}
```

Ticket2: https://github.com/TryGhost/Ghost/issues/26315 (accessed on 1 March 2026)

ORIGINAL

Title:

Drop storing member IP addresses and coordinates on DB

Content:

geolocation column of members table holds many unneeded properties including IP address and coordinates (derived from the IP address) of the members. We only use country, country_code and region properties of that object stored in geolocation column. These "unintended" extra data poses a privacy and security risk for all Ghost sites. Also these "personally identifiable" informations that we collect without consent is against many privacy rules like GDPR.

We should only keep data we need on that column. Also it is needed to have a one-time job that cleans-up those geolocation data previously stored.

REVISED

Title:

Security/Privacy: Drop storing member IP addresses and coordinates in DB

Content:

Description

The geolocation column in the members table currently captures and stores excessive user data, including IP addresses and geographic coordinates. This unintended collection of Personally Identifiable Information (PII) poses significant privacy and security risks and conflicts with data protection regulations such as GDPR.

To ensure compliance and enhance user privacy, the system must be updated to strictly filter the geolocation object before storage, retaining only the country, country_code, and region properties. Additionally, a one-time database migration is required to scrub existing member records and remove any legacy IP addresses and coordinates from the database.

Affected Files (Code Pointers)

ghost/core/core/server/models/member.js: Add a data sanitization step (e.g., in a beforeSave or saving Bookshelf model hook) to strip unneeded properties from the geolocation JSON before inserting/updating.

ghost/core/core/server/services/members/ (or wherever the IP lookup service populates the geolocation object): Update the service to only return the necessary fields.

ghost/core/core/server/data/migrations/versions/ [NEW]: Create a new Knex.js migration script to parse, sanitize, and update the geolocation column for all existing members.

Edge Cases

Migration Performance on Large Sites: Sites with millions of members could experience database locks or timeouts during the migration. The cleanup script must process rows in batches/chunks to ensure smooth execution.

Malformed JSON: Existing geolocation columns might contain null, empty strings, or malformed JSON. The migration script must safely parse and handle these without throwing unhandled exceptions that crash the boot sequence.

Partial Data: A member's geolocation might already be missing an IP but have other fields. The filter should strictly allow-list country, country_code, and region rather than block-listing specific fields.

Domino Effect Analysis

Member Exports: If the CSV export feature was implicitly relying on dumping the entire geolocation object, removing these fields will change the export format. This is desired for privacy, but should be noted in changelogs.

Anti-Spam/Analytics: If any third-party integrations or internal webhooks rely on the database-stored IP address for spam prevention, they will lose access to this historical data. (Note: Live request IPs will still be available at the network layer, just not persisted to the member profile).

Acceptance Criteria

[ ] When a new member signs up or is updated, only country, country_code, and region are saved within the geolocation database column.

[ ] A database migration successfully runs on boot, iterating through existing members and removing ip, latitude, longitude, city, and any other non-essential fields from the geolocation column.

[ ] The migration script uses batching to prevent memory exhaustion or database locking on large instances.

[ ] The Member API and CSV exports no longer expose IP addresses or coordinates.

[ ] Unit tests are added to verify the geolocation object is correctly sanitized before model saves.

Technical Notes (SQL schema/Hooks)

Model Sanitization Logic: In member.js (Bookshelf model):

JavaScript

```
// Inside the saving/beforeSave hook
if (this.hasChanged('geolocation') && this.get('geolocation')) {
    let geo = this.get('geolocation');
    if (typeof geo === 'string') {
        geo = JSON.parse(geo);
```

```
        }

        const sanitizedGeo = {
            country: geo.country || null,
            country_code: geo.country_code || null,
            region: geo.region || null
        };

        this.set('geolocation', JSON.stringify(sanitizedGeo));
    }
```

Migration Strategy: The Knex migration should fetch records where geolocation is not null, parse the JSON, apply the exact same sanitization logic, and perform bulk updates in chunks of ~500–1000 records to maintain performance.

Appendix A.3. Project3: ODOO

https://github.com/odoo/odoo

Ticket1: https://github.com/odoo/odoo/issues/251358

ORIGINAL

Title:

[19.0] Attendance: unable to hide Employee from Attendance/Kiosk Mode

Content:

Steps to Reproduce

Current: all active employee are already in Attendance app and Kiosk Mode

Expected: be able to exclude i a simple manner some Employee from Attendance or Kiosk Mode. Option should be in an Employee detail tab. I want exclude CEO or other Employee not using Attendance app.

REVISED:

Title:

[19.0] Feature: Exclude Employee from Attendance/Kiosk Mode

Content:

Description

Currently, all active employees are automatically fetched and displayed in the Attendance app and Kiosk Mode selection screens. This creates unnecessary UI clutter and privacy concerns for roles that do not require time tracking (e.g., CEO, contractors not paid by the hour).

This ticket implements a toggle on the Employee profile to explicitly exclude specific employees from the Attendance system, hiding them from Kiosk Mode, Barcode/PIN checks, and backend attendance features.

Affected Files (Code Pointers)

hr_attendance/models/hr_employee.py: Add the exclusion field and override state-change methods.

hr_attendance/views/hr_employee_view.xml: Inject the toggle field into the hr_attendance settings section.

hr_attendance/controllers/main.py & hr_attendance/models/res_company.py: Update the search domains that serve the Kiosk mode employee list.

hr_attendance/static/src/public_kiosk/*: Ensure the frontend gracefully handles barcode/PIN errors if an excluded employee attempts to check in.

hr_attendance/static/src/components/*: Update the systray attendance icon logic to hide if the current user's employee profile is excluded.

Acceptance Criteria

Database and UI: A boolean field Exclude from Attendance/Kiosk is available on the Employee form view (under the HR Settings or Attendance tab).

Permissions: The field is only editable by users with the hr_attendance.group_hr_attendance_manager or hr.group_hr_manager roles.

Kiosk Mode Exclusion: When toggled ON, the employee is strictly hidden from the Kiosk Mode employee list (both manual selection and search).

Hardware Rejection: If an excluded employee scans their badge or enters their PIN at a Kiosk, the system must block the action and display a polite error ("Employee not configured for attendance tracking").

Backend Exclusion: The systray "Check In/Out" icon is hidden for the excluded employee when they are logged into the Odoo backend.

Edge Cases

Toggling during an Active Session: An employee is currently checked in, and an admin toggles the exclusion flag to ON.

Handling: Add a constraint or validation hook. The system should either raise a ValidationError ("Cannot exclude an employee with an open attendance session. Please check them out first.") or allow them to check out but block subsequent check-ins. Raising a validation error is the safest data-integrity path.

Multi-Company Environments: Ensure the exclusion respects the standard company_id rules. If an employee is excluded, it applies to all branches they belong to unless specifically tied to a company property (sticking to a global boolean on hr.employee is preferred for simplicity).

Domino Effect Analysis

Timesheets and Payroll (hr_timesheet_attendance): Excluding an employee means they will generate zero attendance hours. HR and Payroll managers must be aware that this skips all attendance-based pay calculations for that employee.

Reports: Excluded employees will not appear in "Missing Attendance" or standard presence reports. Ensure the domains for these reports are updated to [('is_attendance_excluded', '=', False)].

Technical Notes

SQL Schema: Add is_attendance_excluded = fields.Boolean(string="Exclude from Attendance", default=False, tracking=True) to the hr.employee model.

Hooks/Overrides: * Override _attendance_action_change() in hr.employee to raise UserError if is_attendance_excluded == True.

Append ('is_attendance_excluded', '=', False) to the base domain in _get_kiosk_employee _domain (or equivalent controller method feeding the kiosk).

Migration: No data migration script is necessary. The default value False preserves legacy behavior for all existing records.

Ticket2: https://github.com/odoo/odoo/issues/249140 (accessed on 1 March 2026)

ORIGINAL:

Title:

[19.0] module: social

Content:

Steps to Reproduce

In the social marketing app, when creating a post, it is not possible to re-order the pictures. And the user needs to remove and re-upload his pictures in order to have the proper order. This is a big user experience flow.

The ideal would be to have something like the handle widget to order the pictures.

REVISED:

Title:

[19.0] Feature: Allow Image Reordering in Social Posts #249140

Content:

Description

In the Social Marketing app, users currently cannot reorder images once they are attached to a post. To change the sequence, they are forced to delete and re-upload the images in the desired order, resulting in a poor user experience.

This ticket implements a lightweight sequencing mechanism using a standard handle widget, allowing users to drag and drop images into their preferred order directly from the post creation form. Per technical constraints, this will be achieved with minimal architectural overhead and without introducing new complex view types (like custom kanbans).

Affected Files (Code Pointers)

social/models/social_post.py: Update image relational fields to support sequencing.

social/views/social_post_views.xml: Update the form view to include the drag-and-drop handle.

social_facebook/models/social_post_facebook.py (and equivalent social_twitter, social_linkedin, social_instagram files): Ensure API payload generation respects the new image sequence.

Acceptance Criteria

Database and Model: The social post images support a native sequence integer.

UI Implementation: A standard handle widget is visible next to the images in the social.post form view, allowing vertical drag-and-drop reordering.

API Publishing: When a post is published, the images are sent to the respective social media APIs (Facebook, Twitter, etc.) in the exact order defined by the user in the backend.

Backward Compatibility: Existing social posts retain their attached images without throwing relational errors.

Edge Cases

API Limitations: Some social networks (e.g., LinkedIn or Instagram) might have strict constraints on image ordering or limits on media counts per post. The sequencing logic must not interfere with existing API validation checks for maximum allowed images.

Mixed Media: If a post contains a mix of images and a video (if supported by the platform), the sequencing logic should gracefully handle the attachment types without crashing the payload builder.

Domino Effect Analysis

Social API Payloads: The most critical impact is on the _format_images (or equivalent) methods across all social network integration modules. These methods currently iterate over image_ids. They must be updated to ensure they fetch the attachments ordered by the new sequence before converting them to base64 or binary payloads.

Feed Previews: The internal Odoo preview of the social post (how it looks before publishing) must also reflect the updated sequence.

Technical Notes

Schema Update (Minimal Approach): Since a standard Many2many to ir.attachment does not support explicit ordering natively via a handle, we will introduce a very lightweight intermediate model social.post.image containing:

sequence = fields.Integer(default=10)

post_id = fields.Many2one('social.post')

attachment_id = fields.Many2one('ir.attachment', required=True, ondelete='cascade')

Model Refactor: Change image_ids on social.post to be a One2many pointing to social.post.image. (To avoid breaking existing code heavily, you can keep a computed Many2many property that simply returns the attachments ordered by sequence).

View Update: In social_post_views.xml, represent the One2many field as a simple inline <tree editable="bottom">. Add <field name="sequence" widget="handle"/> and the attachment field. Do not create a custom kanban view; stick to the standard list/tree implementation for minimal effort and maximum stability.

Data Migration: Default the sequence to 10. No heavy migration script is needed; existing image_ids can be migrated to the new intermediate table in an init hook based on their default database ID order.

References

1. Hohl, P.; Klünder, J.; van Bennekum, A.; Lockard, R.; Gifford, J.; Münch, J.; Stupperich, M.; Schneider, K. Back to the future: Origins and directions of the "Agile Manifesto"—Views of the originators. *J. Softw. Eng. Res. Dev.* **2018**, *6*, 15. [CrossRef]
2. Forsgren, N.; Humble, J.; Kim, G. *Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations*; IT Revolution: Portland, OR, USA, 2018.
3. Wiegers, K.; Beatty, J. *Software Requirements*, 3rd ed.; Microsoft Press: Redmond, WA, USA, 2013.
4. Cagan, M. *Inspired: How to Create Products Customers Love*; SVPG Press: Sunnyvale, CA, USA, 2008.
5. Perri, M. *Escaping the Build Trap: How Effective Product Management Creates Real Value*, 1st ed.; O'Reilly Media: Sebastopol, CA, USA, 2018.
6. Stettina, C.J.; Heijstek, W. Necessary and Neglected? An Empirical Study of Internal Documentation in Agile Software Development Teams. In *Proceedings of the 2011 International Conference on Software and System Process*; IEEE: New York, NY, USA, 2011; pp. 119–128.
7. Lethbridge, T.C.; Singer, J.; Forward, A. How software engineers use documentation: The state of the practice. *IEEE Softw.* **2003**, *20*, 35–39. [CrossRef]
8. Di Francesco, P.; Lago, P.; Malavolta, I. Architecting with Microservices: A Systematic Mapping Study. *J. Syst. Softw.* **2019**, *150*, 77–97. [CrossRef]
9. Dragoni, N.; Giallorenzo, S.; Lafuente, A.L.; Mazzara, M.; Montesi, F.; Mustafin, R.; Safina, L. Microservices: Yesterday, Today, and Tomorrow. In *Present and Ulterior Software Engineering*; Manuel, M., Bertrand, M., Eds.; Springer: Cham, Switzerland, 2017; pp. 195–216.
10. Hou, X.; Zhao, Y.; Liu, Y.; Yang, Z. Large Language Models for Software Engineering: A Systematic Literature Review. *arXiv* **2023**, arXiv:2308.10620. [CrossRef]
11. Peng, S.; Kalliamvakou, E.; Cihon, P.; Demirer, M. The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. *arXiv* **2023**, arXiv:2302.06590. [CrossRef]
12. Wachnik, B.; Pryciński, P.; Murawski, J.; Nader, M. An analysis of the causes and consequences of the information gap in IT projects. The client's and the supplier's perspective in Poland. *Arch. Transp.* **2021**, *60*, 105–119. [CrossRef]
13. Tan, W.S.; Wagner, M.; Treude, C. Detecting outdated code element references in software repository documentation. *Empir. Softw. Eng.* **2024**, *29*, 5. [CrossRef]
14. Berry, D.M.; Kamsties, E.; Ribeiro, C.; Tjong, S.F. Detecting Defects in Natural Language Requirements Specifications. In *Handbook on Natural Language Processing for Requirements Engineering*; Ferrari, A., Spagnolo, G.O., Eds.; Springer: Cham, Switzerland, 2019; pp. 113–162.
15. Besker, T.; Martini, A.; Bosch, J. Software developer productivity loss due to technical debt—A replication and extension study examining developers' development work. *J. Syst. Softw.* **2019**, *156*, 41–61. [CrossRef]
16. Fernández, D.M.; Wagner, S.; Kalinowski, M.; Felderer, M.; Mafra, P.; Vetrò, A.; Conte, T.; Christiansson, M.-T.; Greer, D.; Lassenius, C.; et al. Naming the pain in requirements engineering: Contemporary problems, causes, and effects in practice. *Empir. Softw. Eng.* **2017**, *22*, 2298–2338. [CrossRef]
17. Boehm, B.W. *Software Engineering Economics*; Prentice-Hall: Englewood Cliffs, NJ, USA, 1981.
18. McConnell, S. *Software Estimation: Demystifying the Black Art*; Microsoft Press: Redmond, WA, USA, 2006.
19. Cohn, M. *Agile Estimating and Planning*; Prentice Hall: Upper Saddle River, NJ, USA, 2005.
20. Graziotin, D.; Fagerholm, F.; Wang, X.; Abrahamsson, P. Consequences of unhappiness while developing software. In Proceedings of the 2nd International Workshop on Emotion Awareness in Software Engineering (SEmotion 17), Buenos Aires, Argentina, 21 May 2017; pp. 42–47.
21. Chikofsky, E.J.; Cross, J.H. Reverse Engineering and Design Recovery: A Taxonomy. *IEEE Softw.* **1990**, *7*, 13–17. [CrossRef]
22. Wason, P.C. On the failure to eliminate hypotheses in a conceptual task. *Q. J. Exp. Psychol.* **1960**, *12*, 129–140. [CrossRef]

23. Stacy, W.; MacMillan, J. Cognitive bias in software engineering. *Commun. ACM* **1995**, *38*, 57–63. [CrossRef]
24. Virk, Y.; Devanbu, P.; Ahmed, T. Calibration of Large Language Models on Code Summarization. *Proc. ACM Softw. Eng.* **2025**, *2*, 2944–2964. [CrossRef]
25. Fan, A.; Gokkaya, B.; Harman, M.; Lyubarskiy, M.; Sengupta, S.; Yoo, S.; Zhang, J.M. Large language models for software engineering: Survey and open problems. In Proceedings of the 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE), Melbourne, Australia, 14–20 May 2023; pp. 31–53.
26. Jiang, J.; Wang, F.; Shen, J.; Kim, S.; Kim, S. A survey on large language models for code generation. *Proc. ACM Softw. Eng.* **2024**, *1*, FSE-00515. [CrossRef]
27. Arkley, P.; Riddle, S. Overcoming the traceability benefit problem. In Proceedings of the 13th IEEE International Requirements Engineering Conference (RE'05), Paris, France, 29 August–2 September 2005; pp. 385–393.
28. Ouf, M.; Li, H.; Zhang, M.; Guizani, M. Reverse Engineering User Stories from Code using Large Language Models. *arXiv* **2025**, arXiv:2509.19587. [CrossRef]
29. Gemini Team Google; Georgiev, P.; Lei, V.I.; Burnell, R.; Bai, L.; Gulati, A.; Tanzer, G.; Vincent, D.; Pan, Z.; Wang, S.; et al. Gemini 1.5: Unlocking Multimodal Understanding Across Millions of Tokens of Context. *arXiv* **2024**, arXiv:2403.05530. [CrossRef]
30. Li, Z.; Li, C.; Zhang, M.; Mei, Q.; Bendersky, M. Retrieval Augmented Generation or Long-Context LLMs? A Comprehensive Study and Hybrid Approach. *arXiv* **2024**, arXiv:2407.16833. [CrossRef]
31. Gim, I.; Chen, Y.; Lee, S.; Suh, Y.; Ahn, S.; Kim, M. Prompt Cache: Modular Attention Reuse for Low-Latency Inference. *arXiv* **2023**, arXiv:2311.04934.
32. Kojima, T.; Gu, S.S.; Reid, M.; Matsuo, Y.; Iwasawa, Y. Large Language Models are Zero-Shot Reasoners. *Adv. Neural Inf. Process. Syst.* **2022**, *35*, 22199–22213.
33. Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Ichter, B.; Xia, F.; Chi, E.; Le, Q.V.; Zhou, D. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Adv. Neural Inf. Process. Syst.* **2022**, *35*, 24824–24837.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.

Reproduced with permission of copyright owner. Further reproduction prohibited without permission.